

ARCHITECTURE AND IMPLEMENTATION REPORT

Structured CSV ingestion

A deliberately small, repeatable, audited path from five source files into PostgreSQL, with clear boundaries between command-line input, application workflow, parsing, and persistence.

IMPLEMENTED / LOCALLY VERIFIED

PROJECT

AI-assisted SupportOps

REPORT DATE

October 1, 2026

SCOPE

Structured CSV import

Operator initiated; service owned

The import is not part of Express startup or an HTTP request. An explicit CLI adapts operator input, then calls a reusable application service. A separate pure parser owns CSV shape and data validation.

EXECUTION Only <code>npm run ingest:csv</code> invokes the import service today.	SERVICE REUSE A future scheduled job or operator workflow may call the service; no such caller is implemented.
ATOMICITY All five business tables and successful audit status commit together.	RELOADS Upsert by source ID. Missing rows in later files are not deleted.

02 / MODULE RESPONSIBILITIES

Three focused boundaries

MODULE	RESPONSIBILITY	CALLED BY
<code>src/cli/import-csv.ts</code>	Parse CLI options, read the five fixed files, calculate raw-byte SHA-256, and print counts/warnings. Check-only validates without opening a DB connection.	Operator via npm script
<code>src/services/csv-import.service.ts</code>	Orchestrate the write import, create/update audit runs, own the transaction, and upsert source rows in foreign-key order.	CLI only, today
<code>src/ingestion/csv-batch-parser.ts</code>	Pure parsing and validation: headers, field types, domains, duplicate IDs, references, ownership, date order, warnings, and physical line provenance.	CLI check mode or import service

Why this is proportionate: it follows the existing service pattern while avoiding a repository layer without a second persistence implementation. SQL remains parameterized; table and column identifiers are fixed in code.

Validate before business writes

- 01 Read fixed input set.** Load five UTF-8 CSV files from the repository `datasets/` directory; calculate one SHA-256 digest per raw file and record the relative source path.
- 02 Parse records.** Use `csv-parse` for quoted values and multiline records. Require exact ordered headers. Record the physical starting line of each data record; the header is line 1.
- 03 Validate the whole batch.** Check required values, enum domains, dates, nullable delivery fields, decimal/integer bounds, duplicate source IDs, case-insensitive customer emails, foreign keys, and ticket/order customer ownership.
- 04 Create durable audit.** A normal write invocation creates a `running` record before validation. Check-only mode creates neither a connection nor an audit row.
- 05 Upsert in dependency order.** Customers, products, orders, order items, then support tickets. Matching IDs are updated; omitted IDs are retained.
- 06 Commit or mark failure.** Business writes and `succeeded` status share a transaction. On failure, roll back the business transaction, then record `failed` separately with a bounded validation message or sanitized `SQLSTATE`.

Audit record

`data_import_runs` stores status and timestamps, per-file source/checksum/row count, target-table counts, quality warnings, and a sanitized failure summary. Migration 004 adds this audit table only; migrations 001–003 and business tables are unchanged. A process interruption can leave a run marked `running`.

04 / OPERATIONAL CONTRACT

Two explicit commands

COMMAND (FROM SERVER/)	EFFECT
<code>npm run ingest:csv -- --check</code>	Read, parse, validate and report; no database connection and no audit record.
<code>npm run ingest:csv</code>	Requires migration 004, creates an audit record, then attempts the atomic upsert.

Known source discrepancy: ORD-1023 has header total 278.99 and line total 228.99, a difference of 50.00 in unspecified currency. Both values are preserved and the difference is a warning, not a rejected row.

Errors identify source file, physical line, and field where available. Database credentials and raw driver messages are not printed or stored in audit summaries. PDF registration/content extraction, chunking, embeddings, vector search, RAG, scheduling, and an HTTP trigger are outside this milestone.

43 source records; repeat import confirmed

8 CUSTOMERS	6 PRODUCTS	10 ORDERS	14 ORDER ITEMS	5 TICKETS
----------------	---------------	--------------	-------------------	--------------

CHECK	OBSERVED RESULT
Local database	Migration 004 is applied. Three successful import runs are recorded with matching per-table counts.
Repeat behavior	Matching source IDs update; a later batch omitting a source row does not delete it.
Failure behavior	Forced database constraint failure rolls back earlier business writes; failed audit survives with sanitized SQLSTATE.
CLI check mode	All 43 source records validate; the ORD-1023 warning is present.
Automated tests	27 backend tests pass, including 9 ingestion parser/service tests.
Static/build gates	Strict TypeScript check and production build pass.
Unrelated tables	One existing application user remains untouched. Document, page, and chunk tables are empty.

Test and build validation used temporary Node 22 because the installed system runtime is Node 18.16.0, below the repository's Vite/Vitest runtime requirement. No global Node version or application engine requirement was changed.